



Domain-Driven Design →

Bruno Santos Fernandes

Edson Cerqueira Junior

Gabriel Lima Brito

Guilherme de Santana Soares Xavier

Jacques Barreto Salah

Lincoln Pereira da Silva

Néphi Santos Luduvico Andrade

Pedro Day Santana França

Samuel Marques Rodrigues de Miranda

Tópicos →

DOMAIN-DRIVEN DESIGN

ENTIDADES

VALUE OBJECTS

DOMAIN SERVICES

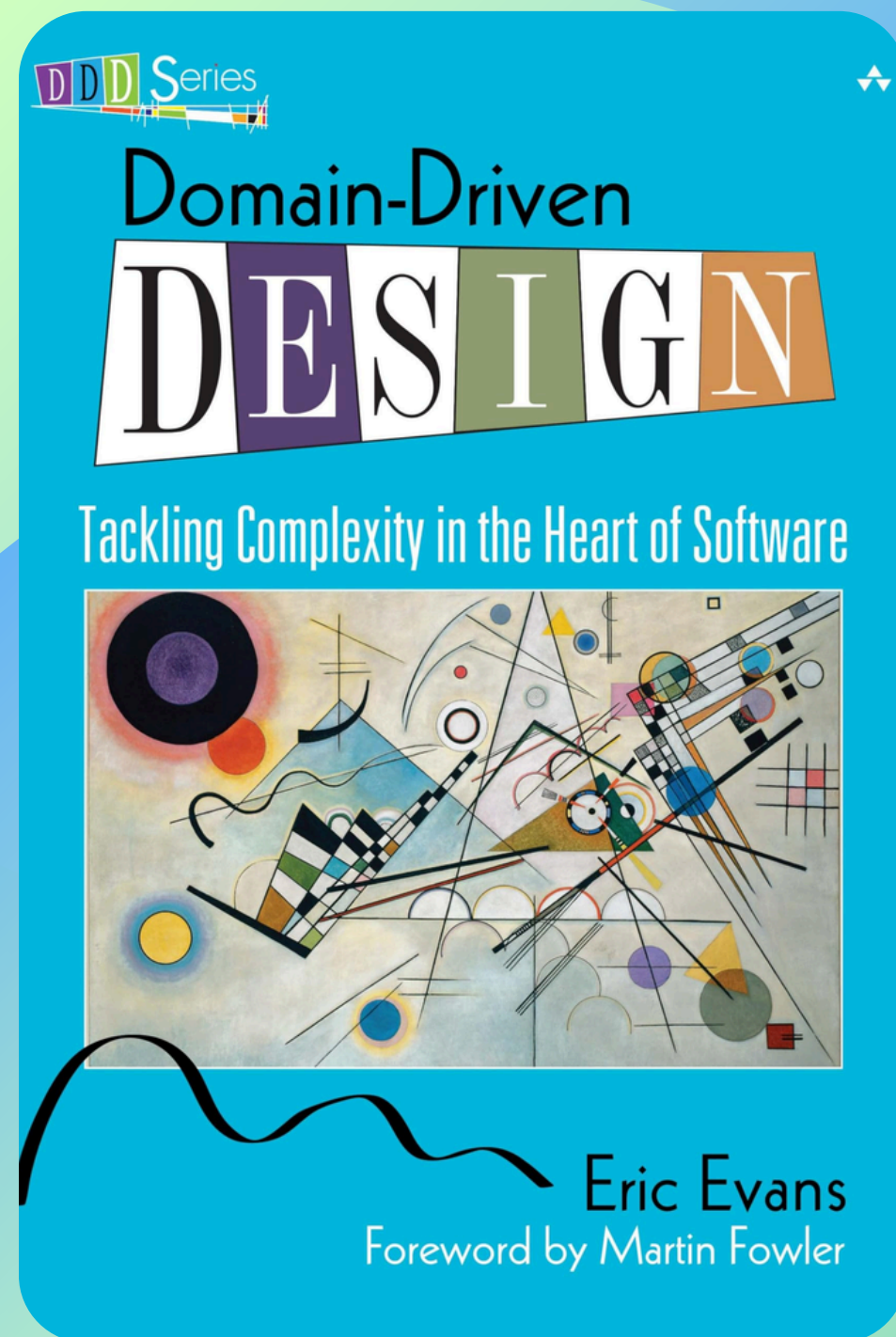
AGGREGATES

FACTORIES E REPOSITORIES

DDD BUROCRÁTICO

CONCLUSÃO





Domain-Driven Design

- Metodologia proposta por Eric Evans (2003);
- Conecta o código ao domínio do negócio;
- Reduz complexidade por meio de modelagem expressiva e colaborativa;
- DDD Estratégico e DDD Tático

Domain-Driven Design Estratégico

- Visão panorâmica do negócio;
- Organiza e divide o domínio em partes gerenciáveis;
- Define fronteiras claras entre equipes e modelos;
- Estrutura como o DDD Tático irá atuar;

Linguagem Ubíqua

- Comum uso de terminologias e jargões técnicos em diferentes ambientes e situações;
- Uma linguagem comum, compartilhada e bem detalhadas que é definida em conjunto com os desenvolvedores e os especialistas do domínio (*Domain Experts*);
- Compartilhada, Explícita e Refletida no código;
- Linguagem Ubíqua determina:
 - os nomes dos elementos do domínio;
 - a definição das entidades;
 - as responsabilidades de cada objeto;
 - sua relação dos elementos do sistema com seus subdomínios.

Subdomínios



<i>Core Domain</i>	É o diferencial competitivo do negócio.
<i>Supporting Subdomain</i>	Fornece funções auxiliares para o core domain.
<i>Generic Subdomain</i>	Área do negócio que é importante para as operações, mas não oferece uma vantagem competitiva.

Bounded Context e Context Map

- **Bounded Context:** fronteira onde o modelo tem significado consistente. Fora de uma fronteira, o mesmo termo pode ter outro significado.
- **Context Map:** documenta como os contextos se relacionam.
 - *Shared Kernel, Customer-Supplier, Conformist, Anti-Corruption Layer (ACL), Open Host Service, Published Language, Separate Ways e Big Ball of Mud*



Domain-Driven Design Tático

A modelagem tática representa a implementação técnica interna de um *bounded context*. Utiliza-se um conjunto de padrões e práticas para transpor o domínio para o código, criando um modelo **rico, expressivo e estritamente alinhado** à linguagem ubíqua.

Entidades:

Em DDD, as entidades representam os objetos do domínio que possuem identidade própria, sendo distinguíveis independente de seus atributos e cumprindo regras para serem consideradas entidades, sendo elas:

- possuir ciclo de vida contínuo mantendo a sua identidade;
- ser capaz de mudar de estado;
- ter identificador Único que define sua identidade;

Estes atributos são essenciais para que as entidades consigam ser manipuladas ao longo de todo o ciclo de vida do software, garantido sua implementação de forma correta dentro das regras de negócio que o sistema necessita.

Exemplo:

```
public class Cliente {
```

```
    private final String cpf;  
    private String nome;  
    private String endereco;  
    private String telefone;  
    private String email;
```

```
    public Cliente(String cpf, String nome, String endereco, String telefone, String email) {
```

```
        if(cpf == null || cpf.isBlank())  
            throw new IllegalArgumentException("CPF inválido!");  
    }
```

```
        if(nome == null || nome.isBlank())  
            throw new IllegalArgumentException("Nome inválido!");  
    }
```

```
    this.cpf = cpf;  
    this.nome = nome;  
    this.endereco = endereco;  
    this.telefone = telefone;  
    setEmail(email);
```

```
}
```

Exemplo:

```
public void atualizarEndereco(String novoEndereco) {  
    if (novoEndereco == null || novoEndereco.isBlank()) {  
        throw new IllegalArgumentException("Endereço inválido");  
    }  
    this.endereco = novoEndereco;  
}  
  
public void atualizarTelefone(String telefone) {  
    this.telefone = telefone;  
}  
  
public void setEmail(String email) {  
    if (email == null || !email.contains("@")) {  
        throw new IllegalArgumentException("Email inválido");  
    }  
    this.email = email;  
}  
  
public String getCpf() {  
    return cpf;  
}  
}
```

Explicação:

O código anterior ilustra uma entidade Cliente, onde ela incorpora como regra de negócio:

- validar CPF;
- impedir email inválido;
- atualizar telefone com regra;
- não permitir nem nome nem CPF vazios.

Value Objects

Um objeto que representa um conceito do domínio sem identidade própria, definido exclusivamente pelos seus atributos, e que encapsula regras e invariantes.

“Sem identidade própria”

Ele não existe como “indivíduo”.

- Um usuário → existe por si só (tem ID)
- Um CPF → não “existe”, ele representa um valor

“Definidos pelos atributos”

Tudo que importa está dentro dele.

- Se dois objetos têm os mesmos valores → são o mesmo conceito.

“Encapsula regras e invariantes”

Ele protege regras do domínio.

Um Email válido:

- tem “@”
- tem formato correto

Imutabilidade

Depois que o objeto é criado, nenhum estado interno pode ser alterado, nem por acidente.

Benefícios:

- o objeto nunca entra em estado inválido depois de criado
- você pode confiar nele em qualquer lugar do sistema

```
class CPF:  
    def __init__(self, valor):  
        self._valor = valor
```

```
@property  
def valor(self):  
    return self._valor
```

```
cpf = CPF("123.456.789-00")  
print(cpf.valor)
```

```
cpf.valor = "987.654.321-00" # tentativa de alteração|
```

```
= RESTART: C:/Users/Luduvico/AppData/Local/Programs/Python/Python313/Scripts/semi  
o de poo imutabilidade.py
```

```
123.456.789-00
```

```
Traceback (most recent call last):
```

```
  File "C:/Users/Luduvico/AppData/Local/Programs/Python/Python313/Scripts/seminar  
e poo imutabilidade.py", line 13, in <module>
```

```
    cpf.valor = "987.654.321-00" # tentativa de alteração
```

```
AttributeError: property 'valor' of 'CPF' object has no setter
```

Ativar o Windows

Accede Configurações para ativar o Win

Igualdade por valor

Dois objetos são iguais se os atributos são iguais, não se são o mesmo objeto na memória.

Se você não implementa igualdade por valor:

- listas e sets se comportam errado
- duplicatas aparecem
- comparações falham silenciosamente

```
class Cor:
    def __init__(self, nome):
        self.nome = nome

    def __eq__(self, other):
        return self.nome == other.nome
```

```
c1 = Cor("azul")
c2 = Cor("azul")
```

```
print(c1 == c2)
```

```
= RESTART: C:/Users/Luduvico/AppData,
o de poo igualdade por valor.py
True
```

Auto-validação

O objeto garante suas próprias regras no momento da criação.

Se não for válido → não existe

```
class Idade:
    def __init__(self, valor):
        if valor < 0:
            raise ValueError("Idade inválida")
        self.valor = valor
```

```
i1 = Idade(20)
print(i1.valor)
```

```
i2 = Idade(-5)
```

```
= RESTART: C:/Users/Luduvico/AppData/Local/Programs/Python/Python39-64/Scripts/python.exe
o de poo auto-validação.py
20
```

```
Traceback (most recent call last):
```

```
  File "C:/Users/Luduvico/AppData/Local/Programs/Python/Python39-64/Scripts/python.exe", line 1, in <module>
    i2 = Idade(-5)
```

```
  File "C:/Users/Luduvico/AppData/Local/Programs/Python/Python39-64/Scripts/python.exe", line 1, in <module>
    i2 = Idade(-5)
```

```
  File "C:/Users/Luduvico/AppData/Local/Programs/Python/Python39-64/Scripts/python.exe", line 1, in <module>
    i2 = Idade(-5)
```

```
  File "C:/Users/Luduvico/AppData/Local/Programs/Python/Python39-64/Scripts/python.exe", line 1, in <module>
    i2 = Idade(-5)
```

```
ValueError: Idade inválida
```

Entidade x Value Object

Entidade:

- Objeto com identidade própria.
- Continua sendo o mesmo objeto mesmo que seus atributos mudem.

Value Object:

- Objeto sem identidade própria.
- É definido apenas pelos seus valores.

Entidade x Value Object

Critério	Entidade	Value Object
Identidade	Possui ID próprio	Não possui ID próprio
Comparação	Pelo ID	Pelos atributos
Mudança de estado	Pode mudar	Deve ser imutável
Exemplo	Cliente, Pedido, Usuário	CPF, Email, Dinheiro, Endereço
Regra principal	Mantém identidade no tempo	Garante validade do valor

Conclusão

Entidade e Value Object
cumprem papéis diferentes no
domínio.

- Entidade é usada quando o objeto precisa de identidade própria e acompanhamento ao longo do tempo.
- Value Object é usado quando o foco está no valor e nas regras do conceito, sem identidade própria.

Essa separação ajuda a:

- modelar melhor o domínio;
- deixar o código mais claro;
- evitar estados inválidos;
- representar as regras de negócio de forma mais fiel.

Domain Services (DS): Definição

- Encapsulam tarefas.
 - DS vs Entidades e Value Objects.
 - Abrigam comportamentos.
 - Sinal de uso correto: operação que se sente deslocada em qualquer entidade.
-

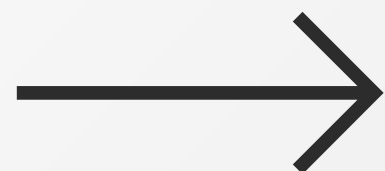
DS: Quando Usar

- Processo de negócio significativo.
 - Transformação entre objetos.
 - Cálculo com múltiplos objetos.
 - Absorção de coordenação.
-

DS: Modelo Anêmico

- Classes de domínio viram contêneiros de dados.
 - Modelo procedural disfarçado de OO.
 - Sinais de alerta:
 - Proliferação indiscrimnada de services.
 - Lógica centralizada nos services.
-

Aggregates



Relações de domínio e
limites de consistência.

Exemplo prático da
definição de agregados.

Exemplo em código.

Invariantes e regras de
fronteira.

Aggregate Roots.

Validação e garantia de
consistência.

Relações de domínio e limites de consistência

Cada relação entre os elementos de um software é regida pelas regras de domínio, que possuem diferentes limites de consistência para cada tipo de relação.

- **Relação de Associação:** quando um elemento (Entidade ou Objeto de Valor) apenas refere-se a outro.
- **Relação de Composição:** quando um elemento é parte essencial de outro.

Os agregados agem como um limite de consistência que envolve somente os elementos ligados pelo mesmo conjunto.

Exemplo prático da definição de agregados

Em um software com as entidades "Cliente", "Pedido" e o objeto "Endereço", a ideia de agregados surge para manter a consistência das relações entre esses elementos. O cliente pode existir independentemente de pedidos, caracterizando uma associação entre "Cliente-Pedido". Já em um cenário onde todo pedido depende de entrega, não é possível existir pedido sem endereço, tornando o "Endereço" parte essencial do pedido, ou seja, uma composição na relação "Pedido-Endereço".

Assim, é possível definir dois agregados: um centrado no "Cliente" e outro centrado em "Pedidos" (incluindo "Endereço"), sendo este último referenciado pelo agregado de cliente por meio de uma relação de referência.

Exemplo de Código →

O exemplo mostra uma modelagem em DDD com dois aggregates: Cliente e Pedido. A relação entre eles é feita por ID, e o Pedido gerencia seus próprios itens internamente.

```
import uuid

class Cliente:
    def __init__(self, nome, tel):
        self.nome = nome
        self.tel = tel
        ClienteID = str(uuid.uuid4())
        self.ID = ClienteID

class Pedido:
    def __init__(self, clienteID, endereco):
        PedidoID = str(uuid.uuid4())
        self.ID = PedidoID
        self.clienteID = clienteID
        self.endereco = endereco
        self.itens = []

    def Adicionar_Item(self, item):
        self.itens.append(item)

class Item_Pedido:
    def __init__(self, nome, quantidade):
        ItemID = str(uuid.uuid4())
        self.nome = nome
        self.quantidade = quantidade
        self.ID = ItemID
```

Invariantes e regras de fronteira

Os agregados garantem os invariantes do domínio, assegurando que regras de negócio permaneçam sempre válidas. As alterações nos seus elementos devem ocorrer de forma controlada dentro de seus limites, preservando a consistência interna. As regras de fronteira determinam que a consistência é mantida apenas dentro do agregado, enquanto a comunicação com outros ocorre por meio de referências.

No exemplo anterior, Cliente deve possuir dados válidos, Pedido deve sempre estar associado a um cliente e conter endereço válido, e Item de Pedido deve ter nome definido e quantidade maior que zero.

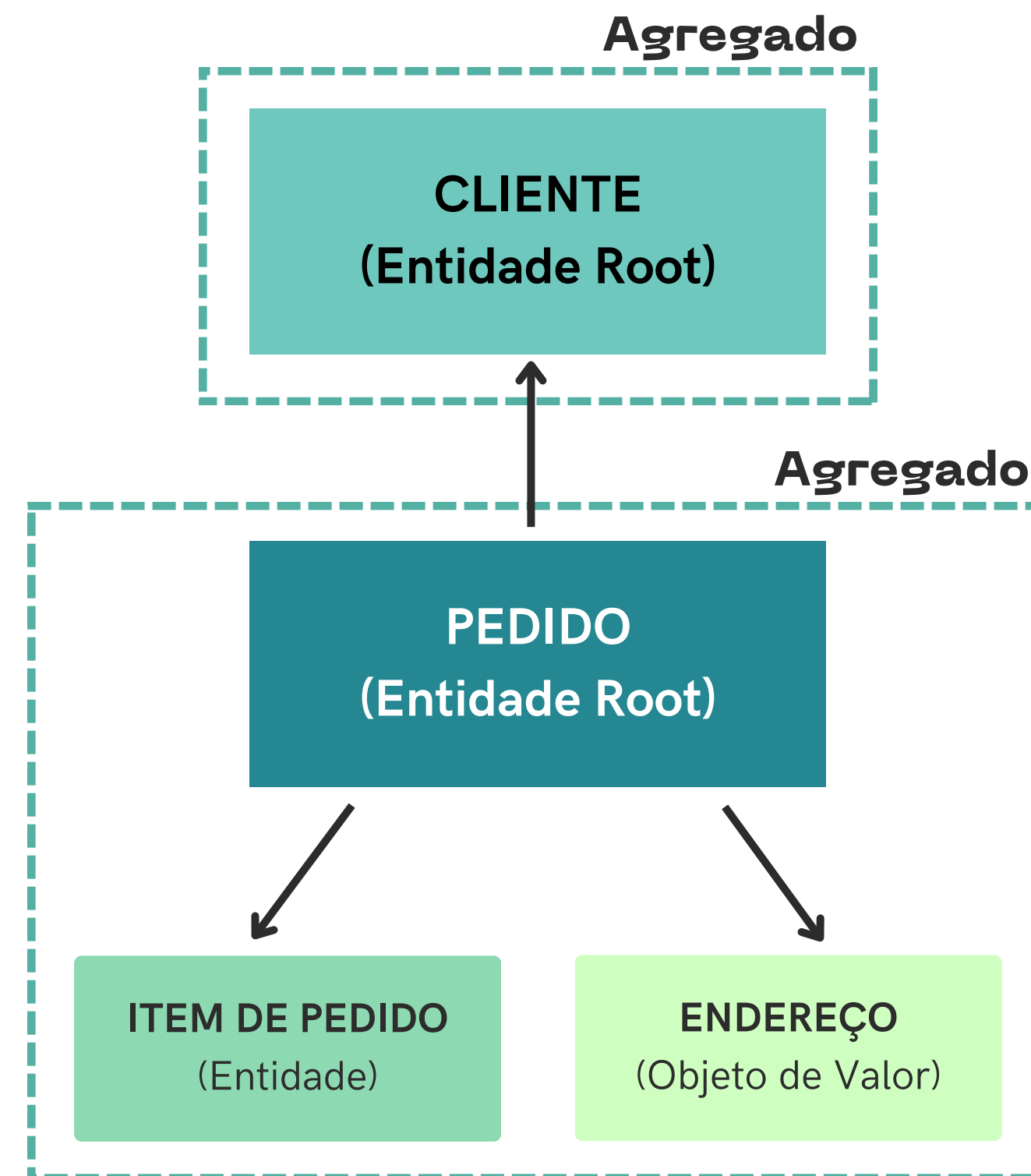
[Voltar ao índice](#)

Aggregate Roots

Outro conceito importante dos agregados são as raízes (aggregate roots).

A raiz é o único elemento do agregado que pode ser acessado externamente, enquanto as demais entidades só podem ser manipuladas através dela.

Esse conceito garante o controle de acesso e evita alterações diretas que possam comprometer a consistência do agregado.



Validação e garantia de consistência

Evans (2003) define o limite de consistência como o conjunto de elementos que deve permanecer válido após alterações no agregado.

As mudanças são validadas e coordenadas pela raiz do agregado, que garante a consistência dos demais elementos internos.



Esse mecanismo evita estados inválidos e assegura o cumprimento das regras de negócio dentro do agregado.

Factories

- Factories são componentes responsáveis por lidar com a criação de objetos de domínio complexos.
- Idéia central: a separação entre criação e comportamento.
- Possuem o benefício de centralizar e encapsular a lógica de criação.
- Garantem o respeito às invariantes do domínio no momento da criação.
- Recomenda-se usar factories apenas para objetos complexos.

Factories

```
import uuid
```

```
class Order:
```

```
    def __init__(self, id, customer, total):
```

```
        self.id = id
```

```
        self.customer = customer
```

```
        self.total = total
```

```
class OrderFactory:
```

```
    @staticmethod
```

```
    def create(customer, total):
```

```
        if total <= 0:
```

```
            raise ValueError("Total inválido")
```

```
        return Order(str(uuid.uuid4()), customer, total)
```

Repositories

- Repositories são responsáveis por persistência e recuperação de objetos.
- Função principal: salvar e buscar objetos no mecanismo de armazenamento.
- No contexto do domínio, se comportam como uma coleção de objetos em memória. Em verdade, se comunicam com o banco de dados sem que o domínio o acesse diretamente (desacoplamento).
- Encapsulam a lógica de armazenamento e busca de objetos.
- Repositories não devem conter regras de negócio, se restringindo ao armazenamento de dados.

Repositories

```
class OrderRepository:
    def __init__(self):
        self.db = {}

    def save(self, order):
        self.db[order.id] = order

    def find_by_id(self, id):
        return self.db.get(id)
```

DDD Burocrático

Apesar das vantagens oferecidas pela aplicação do DDD em projetos, este também possui desvantagens significativas como:

1. **Complexidade:** Conceitos como Entidades, Agregados, Repositórios, etc, tornam o projeto mais difícil de ser escrito.
2. **Verbosidade:** A aplicação do DDD pode acabar aumentando o tamanho do código, assim exigindo mais tempo para se desenvolver.
3. **Over-engineering em CRUDs:** Em aplicações simples, o DDD adiciona camadas a mais sem necessidade.
4. **Dependência de especialistas de domínio:** O sucesso do DDD depende da colaboração dos domain experts (especialistas do negócio) para a construção da linguagem ubíqua.
5. **Impacto na performance:** No DDD, o "Agregado" é carregado completamente, e em modelos mal desenhados, isso pode custar performance.

Conclusão

O DDD é uma escolha estratégica poderosa, mas exige discernimento na aplicação. Seu valor está em conectar profundamente o código ao domínio de negócio, porém, aplicá-lo indiscriminadamente em sistemas simples gera burocracia e *over-engineering* desnecessários.

A chave do sucesso está em concentrar o DDD no *Core Domain*, onde a complexidade justifica o investimento, e evitar sua aplicação em partes triviais do sistema.

Quando bem usado, o DDD deixa de ser apenas uma técnica e se torna uma ponte entre a intenção do negócio e a execução técnica, produzindo sistemas que realmente refletem as necessidades que se propõem a resolver.